ASD1-M1-01 · MINI-PROJEKT M1

Laboratorium sortowania

Interaktywna aplikacja edukacyjna do obserwowania czterech algorytmów na tych samych danych.

Autor	Daniel Nedwecki
Numer indeksu	93797
Przedmiot	Algorytmy i struktury danych I
Rok akademicki	2026/2027
Data	26.10.2026

Dokument zawiera opis zakresu, implementacji, testów, porównania operacji, ograniczeń i źródeł. Dane autora i numer indeksu podano na stronie tytułowej.

02 · CEL I ZAKRES

Problem, odbiorca i wymagania

Celem projektu jest pokazanie, jak cztery metody sortowania przekształcają tę samą tablicę. Odbiorca może śledzić nie tylko wynik, lecz także porównania, przesunięcia, zamiany, podział rekurencyjny i scalanie.

Aplikacja przyjmuje od 0 do 40 liczb całkowitych z zakresu -99…99. Dane mogą zawierać wartości ujemne i duplikaty. Każde wystąpienie ma osobne ID i etykietę, na przykład 5A oraz 5B. Metoda sortuje własną kopię wejścia.

Wykonane metody

• Selection Sort: wybór minimum z nieuporządkowanego sufiksu; zamiana następuje tylko dla różnych pozycji.

• Insertion Sort: przesuwanie większych kluczy i wstawienie elementu w wolne miejsce.

• Merge Sort: rekurencyjny podział oraz scalanie; przy równych kluczach pierwszeństwo ma lewa połowa.

• Quick Sort Lomuto: ostatni element aktywnego fragmentu jest pivotem; widoczna jest granica podziału.

Zakres aplikacji obejmuje edycję tablicy, pięć gotowych scenariuszy, sterowanie krokiem, odtwarzanie, pauzę, reset i tempo. Widok ujawnia indeksy, aktywny fragment, zmienne algorytmu, pseudokod i przyczynę działania.

03 · METODY ITERACYJNE

Selection Sort i Insertion Sort

Selection Sort przechodzi od lewej do prawej. Dla indeksu i znajduje najmniejszą wartość w zakresie i…n-1, a potem umieszcza ją na pozycji i. W tej implementacji pomija zamianę, gdy minimum już znajduje się na właściwej pozycji.

Operacje Selection: każde sprawdzenie A[j] < A[min] zwiększa licznik porównań. Zamiana dwóch różnych pozycji dodaje dwa zapisy. Algorytm wykonuje dokładnie n(n-1)/2 porównań dla n ≥ 1; liczba zapisów zależy od danych.

Insertion Sort utrzymuje uporządkowany prefiks. Bieżący element staje się kluczem. Większe elementy są przesuwane w prawo; klucz trafia w pierwsze wolne miejsce. Porównanie klucza z elementem prefiksu jest liczone, gdy element istnieje.

Stabilność: warunek przesunięcia jest ścisły (wartość > key), dlatego elementy o równych kluczach nie mijają się. Jeśli klucz już zajmuje właściwą pozycję, implementacja nie zapisuje go ponownie.

Koszt zależy od wejścia. Dla tablicy rosnącej Insertion wykonuje n-1 porównań i zero zapisów tablicy. Dla wejścia malejącego wykonuje n(n-1)/2 porównań, przesunięcia oraz zapisy kluczy.

04 · METODY REKURENCYJNE

Merge Sort i Quick Sort Lomuto

Merge Sort dzieli przedział na dwie połowy aż do fragmentów jednoelementowych. Następnie scala uporządkowane połowy do bufora pomocniczego i kopiuje wynik do tablicy. Porównanie z warunkiem <= wybiera lewy element przy równości, co zachowuje stabilność.

Licznik Merge Sort obejmuje zapisy do bufora oraz kopie z bufora do tablicy. Podczas scalania każdy element danego fragmentu zapisuje się raz do bufora i raz z powrotem. Łączna liczba zapisów zależy od rozmiaru i kształtu podziału; mieści się w O(n log n). Pamięć pomocnicza ma rząd O(n).

Quick Sort Lomuto wybiera A[prawy] jako pivot. Zmienna granica wskazuje koniec części wartości <= pivot. Po jednym przejściu pivot umieszczany jest na pozycji między częściami, po czym oba fragmenty są sortowane rekurencyjnie.

Każde porównanie A[j] <= pivot zwiększa licznik porównań. Zamiany różnych pozycji liczą dwa zapisy; gdy granica jest równa j, zbędna zamiana jest pominięta. Zły dobór pivota, na przykład ostatniego elementu w tablicy rosnącej, daje koszt kwadratowy.

Stabilność testuje się po ID, nie po samych wartościach. Selection i Quick Lomuto nie gwarantują zachowania pierwotnej kolejności równych kluczy.

05 · STRUKTURY I INTERFEJS

Od wejścia do rzeczywistej klatki

Model elementu zawiera wartość, niezmienne ID i etykietę widoczną w interfejsie. Funkcja przygotowania danych waliduje rozmiar, typ i zakres, a następnie tworzy kopie dla sortowania.

Moduł algorytmów wykonuje operacje i zapisuje klatki historii w chwili ich wykonania. Klatka przechowuje kopię tablicy, liczniki, linię pseudokodu, aktywny zakres, znaczniki minimum/klucza/pivota, granicę i bufor. Interfejs odtwarza tę historię; nie ma ręcznie nagranej animacji.

Sterowanie

- Krok przesuwa pokaz o jedną klatkę.
- Start/Pauza odtwarza lub zatrzymuje historię.
- Reset wraca do danych wejściowych.
- Suwak zmienia odstęp czasowy od 120 do 1400 ms.
- Edytowanie danych lub zmiana algorytmu tworzy nowy przebieg.

Legenda korzysta jednocześnie z koloru, obramowania i podpisu tekstowego. Oznaczenia aktywnego elementu, minimum, klucza i pivota pozostają czytelne bez rozróżniania samych barw.

06 · TESTY

Testy automatyczne i ich wynik

Testy uruchomiono poleceniem node --test tests/algorithms.test.js. Wynik z lokalnego wykonania: 14 testów zaliczonych, 0 niezaliczonych. Zestaw obejmuje wszystkie metody sortowania, walidację, parsowanie danych i pomiary.

Przypadki sprawdzone

• Każda metoda sortuje wartości mieszane, ujemne i powtarzające się; wszystkie ID pozostają obecne.

• Pusta i jednoelementowa tablica kończą pracę bez porównań kluczy.

• Merge Sort zachowuje kolejność ID równych wartości i etykiety 5A, 5B.

• Insertion Sort zachowuje kolejność elementów o równych kluczach.

• Klatka po znalezieniu nowego minimum w Selection zawiera właściwy indeks i wartość.

• Pierwsza partycja Quick Sort wskazuje ostatni element jako pivot oraz granicę początkową.

• Selection na danych rosnących nie wykonuje zbędnych zamian; liczba zapisów wynosi zero.

• Merge dla wejścia [2, 1] wykonuje jedno porównanie i cztery zapisy: dwa do bufora i dwa z powrotem.

• Walidacja odrzuca 41 elementów, ułamki i wartość spoza zakresu; akceptuje -99 i 99.

• Pomiar tworzy 36 wierszy: 3 rozmiary × 3 układy × 4 metody; wszystkie wyniki są sprawdzane jako uporządkowane.

07 · PORÓWNANIE

Liczniki operacji dla 36 przebiegów

Zestawy wejściowe są deterministyczne. Rosnące i malejące zawierają kolejne wartości; zestaw z powtórzeniami powtarza cykl -2, -1, 0, 1, 2. Każda z czterech metod otrzymuje identyczne wejście dla danego rozmiaru.

C = porównania kluczy. Z = zapisy do tablic, również pomocniczych. Zamiana różnych pozycji to dwa zapisy. Rysowanie, klatki historii i walidacja są wyłączone. To liczniki operacji, nie pomiary czasu zegarowego.

n	Dane	Algorytm	Porównania	Zapisy
10	Rosnące	Selection	45	0
10	Rosnące	Insertion	9	0
10	Rosnące	Merge	19	68
10	Rosnące	Quick Lomuto	45	0
10	Malejące	Selection	45	10
10	Malejące	Insertion	45	54
10	Malejące	Merge	15	68
10	Malejące	Quick Lomuto	45	10
10	Powtórzenia	Selection	45	12
10	Powtórzenia	Insertion	19	14
10	Powtórzenia	Merge	23	68
10	Powtórzenia	Quick Lomuto	29	20
20	Rosnące	Selection	190	0
20	Rosnące	Insertion	19	0
20	Rosnące	Merge	48	176
20	Rosnące	Quick Lomuto	190	0
20	Malejące	Selection	190	20
20	Malejące	Insertion	190	209
20	Malejące	Merge	40	176
20	Malejące	Quick Lomuto	190	20
20	Powtórzenia	Selection	190	30
20	Powtórzenia	Insertion	79	72
20	Powtórzenia	Merge	64	176
20	Powtórzenia	Quick Lomuto	82	60
40	Rosnące	Selection	780	0
40	Rosnące	Insertion	39	0
40	Rosnące	Merge	116	432
40	Rosnące	Quick Lomuto	780	0
40	Malejące	Selection	780	40
40	Malejące	Insertion	780	819
40	Malejące	Merge	100	432
40	Malejące	Quick Lomuto	780	40
40	Powtórzenia	Selection	780	62
40	Powtórzenia	Insertion	319	308
40	Powtórzenia	Merge	164	432
40	Powtórzenia	Quick Lomuto	248	140

Źródło liczb: scripts/benchmark.js uruchomiony na kodzie projektu; pełne dane znajdują się w docs/benchmark-results.json.

08 · INTERPRETACJA

Co pokazują liczniki

Na wejściu rosnącym o długości 40 Insertion wykonuje 39 porównań i zero zapisów. Selection i Quick Lomuto wykonują po 780 porównań. Quick nie musi w tym układzie zapisywać elementów, lecz wybieranie skrajnego pivota nadal wymusza podział 39+0, 38+0 i kolejnych fragmentów.

Na wejściu malejącym o długości 40 Insertion wykonuje 780 porównań i 819 zapisów, ponieważ przesuwa elementy oraz zapisuje klucze. Merge wykonuje 100 porównań i 432 zapisy w przyjętym modelu z buforem.

Na wejściu z powtórzeniami dla n=40 Quick Lomuto wykonuje 248 porównań i 140 zapisów. Wynik pokazuje wpływ rozkładu danych i polityki zamian; nie dowodzi, że ten wariant będzie szybszy dla każdego wejścia.

Selection wykonuje n(n-1)/2 porównań niezależnie od kolejności danych. W badanych rozmiarach rosnących: 45, 190 i 780. Merge zapisuje elementy do bufora i kopiuje je z powrotem, dlatego zapisów jest więcej, lecz podział jest zrównoważony.

Wnioski dotyczą wyłącznie zdefiniowanych liczników, konkretnych tablic i implementacji. Nie wyciągamy z nich wniosków o milisekundach ani szybkości na każdym komputerze.

09 · TECHNOLOGIE I URUCHOMIENIE

Lokalna aplikacja bez kont i API

Interfejs tworzą HTML i CSS; algorytmy oraz sterowanie krokami napisano w JavaScript ES modules. Testy automatyczne wykorzystują wbudowany moduł node:test. Dane pozostają w przeglądarce; projekt nie wykonuje żądań sieciowych do usług AI.

Uruchomienie

1. Zainstaluj Python 3.
2. W katalogu projektu uruchom: python3 -m http.server 8000
3. Otwórz http://127.0.0.1:8000/ w przeglądarce.
4. Testy uruchom poleceniem: npm test.

Pliki źródłowe

• index.html oraz assets/styles.css - układ, typografia i oznaczenia.

• src/algorithms.js - walidacja, cztery algorytmy, historia klatek i liczniki.

• src/app.js - sterowanie, wizualizacja, quiz i tabela porównawcza.

• tests/algorithms.test.js - zestaw testów automatycznych.

• presentation/index.html - osobna prezentacja 10 slajdów.

Użycie AI

Asystent AI pomógł przygotować kod, testy i treści dokumentacji. Autor powinien przeczytać i zrozumieć implementację, samodzielnie powtórzyć testy, zweryfikować wyniki i przedstawić własny wkład podczas obrony.

10 · PODSUMOWANIE I OBRONA

Wynik, ograniczenia i dalsze sprawdzenie

Projekt łączy działające implementacje czterech metod z wizualizacją ich stanów. Testy automatyczne sprawdzają poprawność wyników, stabilność tam, gdzie wymagana, kluczowe stany pośrednie i liczniki zapisów. Porównanie obejmuje 36 deterministycznych przebiegów.

Ograniczenia

• Tablica ma najwyżej 40 liczb całkowitych z zakresu -99…99.

• Quick Sort wykorzystuje Lomuto z ostatnim elementem jako pivotem; może mieć koszt $O(n^2)$.

• Liczniki nie mierzą czasu zegarowego ani pamięci szczytowej.

• Autor samodzielnie opublikuje wymagane repozytorium GitHub.

Plan pokazu

1. Pokaż metodę i pseudokod. 2. Wykonaj jeden przebieg krok po kroku. 3. Zmień dane podczas pokazu. 4. Porównaj liczniki. 5. Pokaż duplikaty lub przypadek pustej tablicy. 6. Odpowiedz na pytanie o bieżący stan. 7. Wyjaśnij wskazany fragment kodu.

Materiały

Princeton Algorithms, Elementary Sorts: algs4.cs.princeton.edu/21elementary/
Princeton Algorithms, Mergesort: algs4.cs.princeton.edu/22mergesort/
Princeton Algorithms, Quicksort: algs4.cs.princeton.edu/23quicksort/

Przed oddaniem: uruchom aplikację na komputerze prezentacyjnym, sprawdź odnośniki, przećwicz demonstrację i samodzielnie opublikuj wymagane repozytorium GitHub.